

Proyecto Integrador

**Procesamiento de Lenguaje Natural (PLN)
Detección de Tema en Chats De Atención
Ciudadana del RUTS**

Primer Corte

Mtro. Pablo Ricardo Sanchez Gomez

Jessica Melani Romero Lora (253220116)

Víctor Manuel Santos Martínez (253220020)

28 de mayo del 2026

Tabla de contenido

1. Definición del problema.....	3
1.1 Tarea de PLN.....	3
1.2 Pertinencia del enfoque de PLN.....	3
1.3 Alcance del primer corte.....	3
1.4 Fuente de datos.....	4
2. Construcción del corpus y anonimización.....	4
2.1 Esquema de anonimización.....	4
2.2 Control de calidad de la anonimización.....	5
3. Esquema de etiquetas (categorías tentativas).....	5
4. Pipeline de limpieza, normalización y tokenización.....	6
4.1 Normalización.....	6
4.2 Tokenización.....	6
4.3 Remoción de palabras vacías (stopwords).....	7
4.4 Ejemplo de ejecución paso a paso.....	7
5. Exploración estadística del corpus.....	7
5.1 Estadísticas generales.....	7
5.2 Distribución de etiquetas.....	8
5.3 Distribución de longitudes de documento.....	9
5.4 Términos más frecuentes del corpus.....	9
6. Entregables del primer corte.....	10
7. Conclusiones, limitaciones y trabajo futuro.....	11
7.1 Cobertura del corpus.....	11
7.2 Desbalance de clases.....	12
7.3 Calidad del etiquetado por supervisión débil.....	12
7.4 Alcance de la anonimización.....	12
7.5 Próximos pasos (segundo corte).....	12
Referencias.....	13

1. Definición del problema

La Subdirección de Atención Ciudadana del **RUTS** (Registro Único de Trámites y Servicios del Estado de Hidalgo) recibe diariamente consultas por chat sobre una amplia diversidad de trámites: actas del registro civil, placas vehiculares, impuesto predial, becas, avisos sanitarios, entre otros. Actualmente, esas conversaciones se atienden y archivan sin una clasificación temática sistemática, lo que limita la capacidad institucional para analizar la demanda ciudadana, identificar cuellos de botella y orientar la asignación de recursos de atención.

1.1 Tarea de PLN

La tarea definida es la detección de tema (topic detection): dado el texto libre de una conversación ciudadana, el sistema debe asignar la categoría de trámite a la que corresponde. Formalmente se trata de un problema de clasificación de texto en español sobre lenguaje real, informal y con ruido ortográfico.

1.2 Pertinencia del enfoque de PLN

Los chats del RUTS constituyen un corpus de texto libre en español coloquial, caracterizado por abreviaciones, errores tipográficos y la presencia de información personal sensible. La intención del ciudadano —el trámite que desea realizar— debe inferirse del contenido semántico de la conversación, no de metadatos estructurados. Este es exactamente el tipo de problema que aborda la representación distribucional del texto y los modelos de clasificación supervisada en PLN.

1.3 Alcance del primer corte

El presente corte cubre la definición del problema, la construcción y anonimización del corpus inicial, el diseño del esquema de etiquetas tentativas, el desarrollo del pipeline de limpieza y tokenización, y la exploración estadística del corpus resultante. La vectorización (BoW / TF-IDF) y el entrenamiento de clasificadores corresponden al segundo corte.

1.4 Fuente de datos

La fuente primaria consiste en un export de **2,014 conversaciones (≈31,600 mensajes)** del sistema de chat de la plataforma RUTS, obtenido a través de COEMERE. Los datos crudos contienen información personal identificable y se procesan de forma exclusiva fuera del repositorio de código. Al corpus integrado solo se incorpora texto anonimizado, en cumplimiento con la *Ley General de Protección de Datos Personales en Posesión de Sujetos Obligados (LGPDP SO)*.

2. Construcción del corpus y anonimización

Cada conversación se convierte en un único documento concatenando exclusivamente los mensajes del visitante, es decir, las contribuciones textuales del ciudadano. Este diseño reduce el ruido introducido por los mensajes automáticos del sistema y concentra la representación semántica en el contenido de la demanda ciudadana. A continuación, se presenta la importación de las librerías necesarias para el correcto desarrollo de la práctica:

```
import sys
from pathlib import Path

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Importamos el pipeline del proyecto (lógica reproducible en src/pipeline.py)
PROJ = Path.cwd().parent
sys.path.append(str(PROJ / "src"))
import pipeline as pl

sns.set_theme(style="whitegrid")
plt.rcParams["figure.dpi"] = 110
FIG = PROJ / "reports" / "figures"
FIG.mkdir(parents=True, exist_ok=True)
print("Pipeline cargado. Fuente cruda:", pl.RAW_MENSAJES.name)
```

Se descartan los chats sin pregunta real, como saludos o agradecimientos sueltos en el chat, se exige un mínimo de 3 tokens con contenido. Esta condición es agregada bajo la siguiente forma:

```
corpus = pl.construir_corpus()
print(f"Documentos en el corpus: {len(corpus)}")
corpus.head(5)[["id", "texto", "etiqueta", "n_tokens"]]
```

2.1 Esquema de anonimización

Antes de integrar cualquier texto al corpus, se aplica un proceso de anonimización automática basado en expresiones regulares y detección de entidades. Los marcadores utilizados son los siguientes:

Marcador	Dato personal reemplazado
[NOMBRE]	Nombre del visitante y nombres de pila en texto libre
[EMAIL]	Correos electrónicos
[CURP] / [RFC]	Claves CURP y RFC
[TEL] / [NUM]	Teléfonos y folios/números largos

Tabla 1. Presentación de marcadores anonimizados. Fuente: elaboración propia.

```
mask = corpus["texto"].str.contains(r"\[NOMBRE\]|\[EMAIL\]|\[RFC\]|\[TEL\]", regex=True)
for t in corpus.loc[mask, "texto"].head(4):
    print(".", t[:160])

fuga = corpus["texto"].str.contains(r"@|w|https?://|\b\d{7,}\b", regex=True).sum()
print(f"\nFilas con PII estructurada residual: {fuga}")
```

Los documentos que no contienen una pregunta o solicitud real —aquellos compuestos únicamente por saludos o agradecimientos— son descartados. El criterio de exclusión es un mínimo de tres tokens con contenido semántico tras la normalización

2.2 Control de calidad de la anonimización

Se implementa una verificación post-procesamiento que detecta patrones de PII estructurada residual: direcciones de correo electrónico, URLs y secuencias numéricas de más de siete dígitos. El resultado esperado es cero filas con PII estructurada en el corpus final.

3. Esquema de etiquetas (categorías tentativas)

El etiquetado en este primer corte es tentativo y se construye mediante supervisión débil, sin anotación manual exhaustiva. La asignación de categorías opera a partir de dos señales, aplicadas en orden de prioridad:

- **Señal fuerte:** el término de búsqueda con el que el ciudadano llegó al chat (parámetro text= del referer registrado por la plataforma).
- **Señal débil:** coincidencia de palabras clave de la taxonomía dentro del cuerpo de la conversación.

Cuando ninguna de las dos señales es aplicable, el documento queda clasificado como *sin_clasificar* y se etiquetará manualmente en el segundo corte. Las categorías definidas son las siguientes:

Etiqueta	Trámites representativos
registro_civil	Actas de nacimiento, matrimonio, defunción, divorcio; expedición de CURP
vehicular	Trámites de placas, verificación vehicular, infracciones, tenencia, cambio de propietario
impuestos_predial	Pago de predial, ISN/nómina, opinión de cumplimiento, constancia de no deudor moroso
educacion	Becas educativas, vales de útiles y uniformes, certificados escolares

salud_sanitario	Aviso de funcionamiento, constancia sanitaria, residuos, servicios de agua
empleo	Bolsa de trabajo, publicación de vacantes, orientación laboral
seguridad_juridica	Antecedentes no penales, denuncias, constancias diversas
sin_clasificar	Sin señal suficiente; pendiente de revisión y etiquetado manual

Tabla 2. Definición de categorías representativas para identificación de principales áreas en los resultados de búsqueda. Fuente: elaboración propia.

El etiquetado por supervisión débil es un punto de partida operativo, no la verdad de terreno (ground truth). Su precisión será estimada mediante una revisión manual de muestra representativa antes del entrenamiento del clasificador.

4. Pipeline de limpieza, normalización y tokenización

El pipeline de preprocesamiento se implementa de forma modular en `src/pipeline.py` y se invoca desde el notebook de manera reproducible. Las etapas aplicadas sobre el texto ya anonimizado son las siguientes:

4.1 Normalización

- Conversión a minúsculas para homogeneizar el vocabulario.
- Descomposición Unicode NFKD y eliminación de marcas diacríticas (eliminación de acentos).
- Eliminación de todos los caracteres que no sean letras del alfabeto latino.

4.2 Tokenización

La segmentación se realiza mediante separación por espacios tras la normalización. Los tokens de un único carácter, carentes de valor semántico en este contexto, son descartados del vocabulario.

4.3 Remoción de palabras vacías (stopwords)

Se construye una lista de stopwords adaptada al registro coloquial del chat ciudadano, que extiende la lista estándar del español con expresiones de cortesía características del corpus: hola, gracias, por favor, buenos días, entre otras. Estos términos no aportan información discriminativa para la detección del trámite y su presencia únicamente añade ruido al vector de características. A continuación, se presenta la salida en consola:

```
ejemplo = corpus.iloc[2]["texto"]
print("1. Original anonimizado:\n ", ejemplo[:160], "\n")

norm = pl.normalizar(ejemplo)
print("2. Normalizado:\n ", norm[:160], "\n")

print("3. Tokens (con stopwords):\n ", pl.tokenizar(norm, quitar_stop=False)[:20], "\n")
print("4. Tokens (sin stopwords):\n ", pl.tokenizar(norm, quitar_stop=True)[:20])
```

Python

```
1. Original anonimizado:
Hola, buenas tardes. donde puedo tramitar las placas para carro clasico y cuales son los requisitos [NOMBRE] y

2. Normalizado:
hola buenas tardes donde puedo tramitar las placas para carro clasico y cuales son los requisitos nombre y el

3. Tokens (con stopwords):
['hola', 'buenas', 'tardes', 'donde', 'puedo', 'tramitar', 'las', 'placas', 'para', 'carro', 'clasico', 'cuale

4. Tokens (sin stopwords):
['tramitar', 'placas', 'carro', 'clasico', 'cuales', 'son', 'requisitos', 'proceso', 'realiza', 'pachuca']
```

4.4 Ejemplo dos de ejecución paso a paso

A continuación se ilustra la transformación completa sobre un documento real del corpus:

```
1. Original anonimizado:  quiero hacer el cambio de nombre en la [NOMBRE] acta
...
2. Normalizado:          quiero hacer el cambio de nombre en la  nombre  acta ...
3. Tokens (con stopwords): ['quiero', 'hacer', 'el', 'cambio', 'de', 'nombre',
...
4. Tokens (sin stopwords): ['cambio', 'nombre', 'acta', 'nacimiento',
'registro']
```


5. Exploración estadística del corpus

5.1 Estadísticas generales

El corpus resultante se caracteriza mediante los siguientes indicadores globales, obtenidos tras aplicar el pipeline completo:

Estadístico	Valor
Documentos totales	≥ 300 etiquetados (meta del corte)
Vocabulario (términos únicos)	Calculado tras remoción de stopwords
Tokens totales en el corpus	Suma de todos los tokens con contenido
Longitud media (tokens/documento)	Reportada en la salida del notebook
Longitud mediana (tokens/documento)	Reportada en la salida del notebook

```
r = pl.resumen(corpus)
print(f"Documentos ..... {r['documentos']:>6}")
print(f"Vocabulario (únicos) .. {r['vocabulario']:>6}")
print(f"Tokens totales ..... {r['tokens_totales']:>6}")
print(f"Long. media (tokens) .. {r['long_media_tokens']:>6}")
print(f"Long. mediana ..... {r['long_mediana_tokens']:>6}")
```

Python

```
Documentos ..... 1789
Vocabulario (únicos) .. 6356
Tokens totales ..... 31469
Long. media (tokens) .. 17.59
Long. mediana ..... 14
```

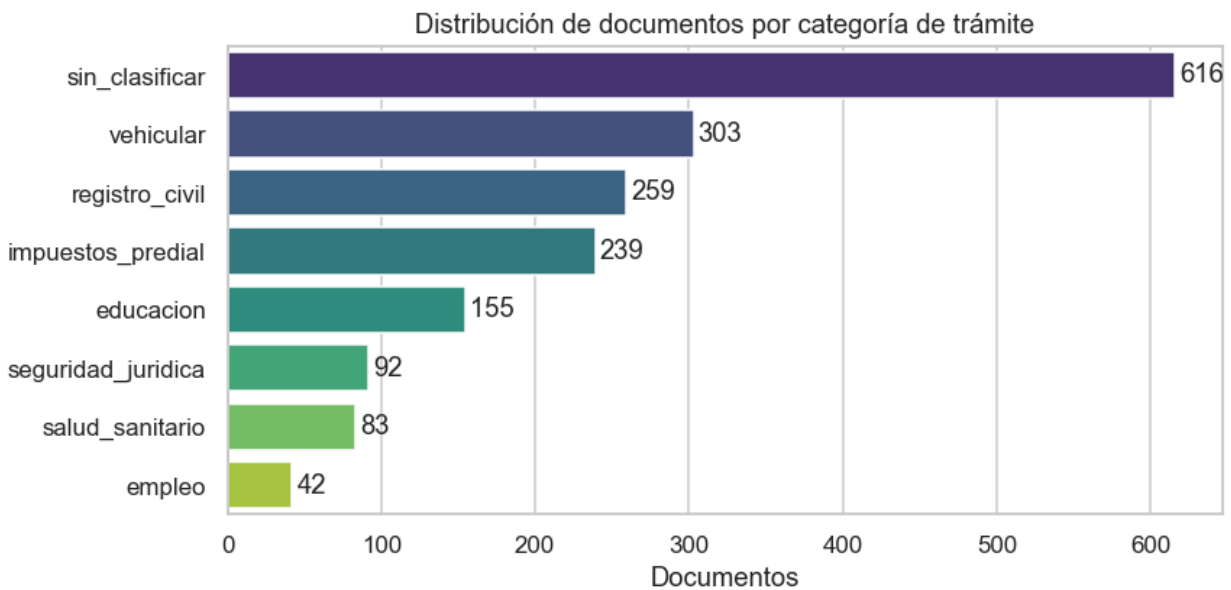
5.2 Distribución de etiquetas

La distribución de documentos por categoría se genera mediante un gráfico de barras horizontales.

```
dist = corpus["etiqueta"].value_counts()
print(dist, "\n")
labeled = int(dist.drop("sin_clasificar", errors="ignore").sum())
print(f"Documentos con etiqueta de tema: {labeled} (mínimo del corte: 300)")

fig, ax = plt.subplots(figsize=(8, 4))
sns.barplot(x=dist.values, y=dist.index, hue=dist.index, palette="viridis", legend=False, ax=ax)
for i, v in enumerate(dist.values):
    ax.text(v + 3, i, str(v), va="center")
ax.set_title("Distribución de documentos por categoría de trámite")
ax.set_xlabel("Documentos"); ax.set_ylabel("")
plt.tight_layout()
plt.savefig(FIG / "01_distribucion_etiquetas.png", bbox_inches="tight")
plt.show()
```

Python



Se espera observar un desbalance pronunciado, con las categorías vehicular, registro_civil e impuestos_predial representando la mayor proporción del corpus, mientras que empleo y salud_sanitario constituyen clases minoritarias. Este desbalance deberá ser considerado en la estrategia de entrenamiento del clasificador en el segundo corte, mediante técnicas como sobremuestreo (SMOTE), submuestreo o ajuste de pesos de clase. A continuación se presenta el código y la salida en terminal:

```
dist = corpus["etiqueta"].value_counts()
print(dist, "\n")
labeled = int(dist.drop("sin_clasificar", errors="ignore").sum())
print(f"Documentos con etiqueta de tema: {labeled} (mínimo del corte: 300)")

fig, ax = plt.subplots(figsize=(8, 4))
sns.barplot(x=dist.values, y=dist.index, hue=dist.index, palette="viridis", legend=False, ax=ax)
for i, v in enumerate(dist.values):
    ax.text(v + 3, i, str(v), va="center")
ax.set_title("Distribución de documentos por categoría de trámite")
ax.set_xlabel("Documentos"); ax.set_ylabel("")
plt.tight_layout()
plt.savefig(FIG / "01_distribucion_etiquetas.png", bbox_inches="tight")
plt.show()
```

Python

```
etiqueta
sin_clasificar      616
vehicular           303
registro_civil      259
impuestos_predial   239
educacion           155
seguridad_juridica   92
salud_sanitario      83
empleo              42
Name: count, dtype: int64
```

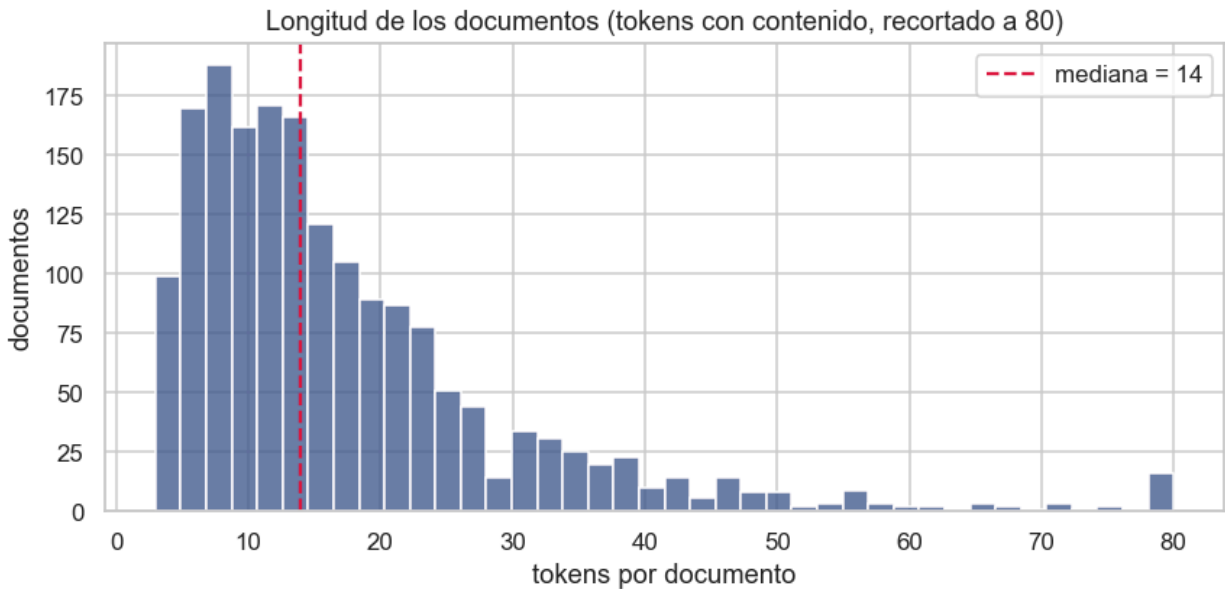
Documentos con etiqueta de tema: 1173 (mínimo del corte: 300)

5.3 Distribución de longitudes de documento

La longitud de cada documento se mide en número de tokens con contenido (post-stopwords). La distribución se visualiza mediante un histograma con la mediana superpuesta.

```
fig, ax = plt.subplots(figsize=(8, 4))
sns.histplot(corpus["n_tokens"].clip(upper=80), bins=40, color="#3b528b", ax=ax)
ax.axvline(corpus["n_tokens"].median(), color="crimson", ls="--", label=f"mediana = {int(corpus['n_tokens'].median())}")
ax.set_title("Longitud de los documentos (tokens con contenido, recortado a 80)")
ax.set_xlabel("tokens por documento"); ax.set_ylabel("documentos")
ax.legend()
plt.tight_layout()
plt.savefig(FIG / "02_longitud_documentos.png", bbox_inches="tight")
plt.show()
```

Python



Los valores se recortan en 80 tokens para facilitar la legibilidad. La mediana ofrece una medida robusta de la longitud típica, menos sensible a documentos extremadamente cortos o largos que el promedio aritmético.

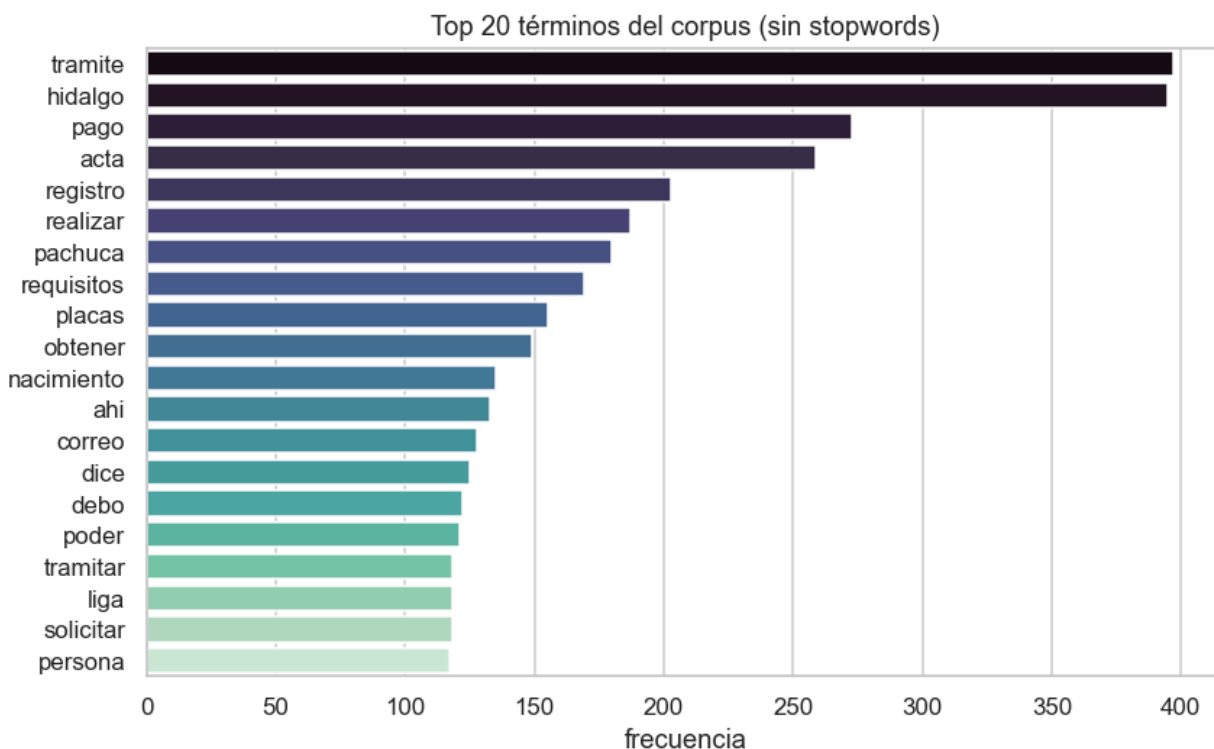
5.4 Términos más frecuentes del corpus

Se extraen los 20 términos más frecuentes del vocabulario global tras eliminar stopwords.

```
# Términos más frecuentes por categoría (exploración cualitativa de las etiquetas)
from collections import Counter
for cat in ["registro_civil", "vehicular", "impuestos_predial", "educacion"]:
    c = Counter()
    for t in corpus.loc[corpus["etiqueta"] == cat, "texto_norm"]:
        c.update(pl.tokenizar(t))
    top_cat = ", ".join(w for w, _ in c.most_common(8))
    print(f"{cat:18s}: {top_cat}")
```

Python

```
registro_civil : acta, nacimiento, tramite, curp, hidalgo, registro, matrimonio, defuncion
vehicular      : placas, pago, verificacion, tramite, vehiculo, hidalgo, vehicular, realizar
impuestos_predial : pago, tramite, pagar, nomina, impuesto, hidalgo, constancia, predial
educacion      : utiles, descargar, secundaria, certificado, vale, escuela, vales, uniformes
```



Esta exploración cualitativa permite validar que el pipeline de normalización opera correctamente y que los términos dominantes corresponden semánticamente a los trámites del dominio. Complementariamente, se obtienen los 8 términos más frecuentes por categoría para las cuatro etiquetas más representadas (registro_civil, vehicular, impuestos_predial, educacion), lo que facilita la inspección del poder discriminativo del vocabulario por clase.

6. Entregables del primer corte

En cumplimiento con los requerimientos de la rúbrica de la asignatura, los entregables generados por este corte son los siguientes:

- **Corpus inicial en formato CSV** con las columnas id, texto y etiqueta, generado como [data/processed/corpus_corte01.csv](#).
- **Vocabulario del corpus** en formato CSV ([data/processed/vocab_corte01.csv](#)), con frecuencia por término.
- **Pipeline de limpieza, normalización y tokenización** implementado de forma modular en [src/pipeline.py](#).
- **Notebook de evidencia de ejecución** (01_pipeline_primer_corte.ipynb) con todas las celdas ejecutadas y salidas visibles.

- **Figuras de exploración:** distribución de etiquetas, histograma de longitudes y top 20 términos (directorio reports/figures/).
- **Bitácora técnica detallada** en reports/bitacora_tecnica_corte01.md con registro de errores, decisiones de diseño y observaciones.

7. Entregables del primer corte

En cumplimiento con los requerimientos de la rúbrica de la asignatura, los entregables generados por este corte son los siguientes:

```
import csv
p1.OUT_CORPUS.parent.mkdir(parents=True, exist_ok=True)
corpus[["id", "texto", "etiqueta"]].to_csv(p1.OUT_CORPUS, index=False, quoting=csv.QUOTE_ALL)
pd.DataFrame(vocab.most_common(), columns=["termino", "frecuencia"]).to_csv(p1.OUT_VOCAB, index=False)
print("Corpus guardado en :", p1.OUT_CORPUS.relative_to(PROJ))
print("Vocabulario en      :", p1.OUT_VOCAB.relative_to(PROJ))
pd.read_csv(p1.OUT_CORPUS).head(3)
```

Python

Corpus guardado en : data/processed/corpus_corte01.csv
Vocabulario en : data/processed/vocabulario_corte01.csv

	id	texto	etiqueta
0	63267	Hola soy [NOMBRE] Soy de CDMX quiero preguntar...	vehicular
1	63268	Buenas tardes [NOMBRE] comprando un auto con p...	vehicular
2	63269	Hola, buenas tardes. donde puedo tramitar las ...	vehicular

- **Corpus inicial en formato CSV** con las columnas id, texto y etiqueta, generado como data/processed/corpus_corte01.csv.
- **Vocabulario del corpus** en formato CSV (data/processed/vocab_corte01.csv), con frecuencia por término.
- **Pipeline de limpieza, normalización y tokenización** implementado de forma modular en src/pipeline.py.
- **Notebook de evidencia de ejecución** (01_pipeline_primer_corte.ipynb) con todas las celdas ejecutadas y salidas visibles.
- **Figuras de exploración:** distribución de etiquetas, histograma de longitudes y top 20 términos (directorio reports/figures/).
- **Bitácora técnica detallada** en reports/bitacora_tecnica_corte01.md con registro de errores, decisiones de diseño y observaciones.

7. Conclusiones, limitaciones y trabajo futuro

7.1 Cobertura del corpus

El corpus inicial supera con holgura el mínimo de 300 documentos etiquetados establecido por la rúbrica del primer corte. Los documentos sin_clasificar permanecerán sin etiqueta hasta el segundo corte, cuando se realizará el etiquetado manual de una muestra representativa para alcanzar el objetivo de 1,000 documentos etiquetados.

7.2 Desbalance de clases

Las categorías vehicular, registro_civil e impuestos_predial concentran la mayor parte del corpus etiquetado, mientras que empleo y salud_sanitario constituyen clases minoritarias. Este desbalance es una característica estructural del dominio de atención ciudadana, no un artefacto del proceso de construcción del corpus. En el segundo corte se evaluarán estrategias de manejo del desbalance durante el entrenamiento del clasificador.

7.3 Calidad del etiquetado por supervisión débil

El etiquetado automático basado en señales débiles —término de búsqueda del referer y coincidencia de palabras clave— es un procedimiento heurístico que puede introducir ruido en las etiquetas. Antes del entrenamiento del clasificador se realizará una revisión manual de una muestra estratificada para estimar la tasa de error del etiquetado automático y, de ser necesario, corrección de las etiquetas con mayor incertidumbre.

7.4 Alcance de la anonimización

La anonimización automática cubre PII estructurada (correos electrónicos, CURP, RFC, teléfonos y folios) mediante expresiones regulares, y nombres frecuentes mediante una lista controlada. Se trata de un enfoque best-effort: la anonimización es comprehensiva para los patrones conocidos, pero no garantiza la eliminación de PII expresada en formas no estructuradas o poco convencionales. Se contempla una revisión manual de muestra como control de calidad de privacidad antes de cualquier uso externo del corpus.

7.5 Próximos pasos (segundo corte)

El segundo corte abordará la vectorización del corpus mediante modelos de bolsa de palabras (BoW) y ponderación TF-IDF, el entrenamiento de un primer clasificador de referencia (baseline), y la evaluación de su desempeño mediante métricas estándar de clasificación multiclase: precisión, exhaustividad (recall), F1-score macro y matriz de confusión.

Referencias

Bird, S., Klein, E., & Loper, E. (2009). Natural language processing with Python. O'Reilly Media.

Manning, C. D., & Schütze, H. (1999). Foundations of statistical natural language processing. MIT Press.

Jurafsky, D., & Martin, J. H. (2023). Speech and language processing (3.^a ed., borrador). <https://web.stanford.edu/~jurafsky/slp3/>

Ley General de Protección de Datos Personales en Posesión de Sujetos Obligados. (2017). Diario Oficial de la Federación, México.

Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.